

OPTICAL CHARACTER RECOGNITION AND TRANSCRIPTION OF LOW-RESOURCE LANGUAGES

JAN PAWEŁ MUSIOŁ

JAMUS23@STUDENT.SDU.DK

SUPERVISED BY

APARNA S. VARDE

APVA@MMMI.SDU.DK

JUAN HEREDIA

JEHM@MMMI.SDU.DK

Bachelor Project in Software Engineering

May 2026



The Maersk Mc-Kinney Moeller Institute
University of Southern Denmark

Abstract

North Sámi is an indigenous Uralic low-resource language, classified by UNESCO as *definitely endangered* and faces many challenges in digitalization. The most accurate publicly available Optical Character Recognition (OCR) models for the language are large transformer based variants released by the National Library of Norway (NLN), their 350 MB size and over 12 seconds per image inference time on CPU make them unsuitable for the community driven heritage projects, mobile field documentation and unbudgeted institutions that would benefit most from them.

This bachelor thesis explores whether a much smaller architecture can approach the accuracy of the state of the art (SOTA) model closely enough to be useful. A modular OCR pipeline (5 backbones $\times$ 3 sequence encoders, with a shared CTC head) was built. Only four variants were benchmarked (due to resource constraints), alongside seven public Transformer OCR (TrOCR) variants, on a 1,048 sample test set covering both synthetic in domain text and historical out of domain content from Turi’s 1910 *Muitalus sámiid birra*.

The headline contribution, `ctc_simple`, is a 7 layer Convolutional Neural Network—Connectionist Temporal Classification (CNN-CTC) model trained from scratch. At 23 MB and 0.033 s per image on CPU, it reaches 12.24% CER (Character Error Rate) (87.72% character accuracy), within 3.69 pp in CER of the best transformer baseline 8.55% CER (91.56% character accuracy) while running roughly $380\times$ faster and being $15\times$ smaller. A linguistically motivated conjunction based sentence splitting step further reduces long line CER from 15.17% to 8.12%, even if only as a proof of concept, and an end to end OCR→Machine Translation (MT) pipeline against TartuNLP’s North Sámi English API demonstrates the model’s usability for downstream tasks.

This summary report focuses on the project’s context, goals, process and process reflections, providing more detailed background and a personal perspective on the work. The technical details, experiments and scientific claims are covered in the full conference paper accepted to IISA 2026, titled **Lightweight OCR in North Sámi Heritage Digitization for Smart Learning**, and in the extended abstract accepted to SCAI 2026, titled **OCR and Machine Translation Pipeline for Low Resource Sámi Languages**.

Contents

Contents	ii
1 Context	1
1.1 Document Scope	1
1.2 Sámi Languages	1
1.3 Linguistic Characteristics of North Sámi	3
1.4 Personal Motivation	5
1.5 Technical Context	5
2 Goals	6
2.1 Research Question	7
2.2 Specific Goals	7
3 Process	8
3.1 Dataset Exploration	8
3.2 Corpus Building Detour	9
3.3 Training Pipeline Overview	10
3.4 Comparison	10
3.5 The lightweight CNN-CTC variant	11
3.6 Training Setup	13
3.7 Evaluation and End to End Pipeline	13
3.8 Conjunction Splitting Experiment	13
4 Process Reflections	16
4.1 Main Results	16
4.2 Out of Domain Generalisation	19
4.3 Conjunction Based Sentence Splitting	20
4.4 End to End Pipeline	20
4.5 Applications	21
4.6 Future work	22
4.7 Conclusion	22
A Background Concepts and Methods	24
A.1 Convolutional Neural Networks	24
A.2 VGG16 / VGG19	25
A.3 ResNet50 / ResNet101	25
A.4 ImageNet Pretraining and Transfer Learning	25
A.5 BiLSTM	25

A.6 Transformer Encoder	26
A.7 TrOCR	26
A.8 Connectionist Temporal Classification (CTC)	26
A.9 AdamW Optimizer	27
A.10 Learning Rate (LR), LR Schedulers and Weight Decay	27
A.11 Epochs, Batch Size, Training Steps, and Early Stopping	27
A.12 Data Augmentation	28
A.13 OCR Evaluation Metrics	28
A.14 Machine Translation Metrics	28
A.15 Statistical Significance and Effect Size	29
B Appendix	30
B.1 Code and Models	30
B.2 Conference Submissions	30
B.3 Supervisor and Meeting Log	30
B.4 AI usage	30
Bibliography	32

1 Context

1.1 Document Scope

This is a summary report covering goals, context, process and process reflections of a Bachelor Thesis project. The project explores different approaches to Optical Character Recognition (OCR) - transcribing images of text into computer readable strings (e.g. `.png`→`.txt`) for North Sámi, an endangered language in the Sámi branch of the Uralic family. The motivation is to contribute to the preservation of Sámi cultural heritage by developing a more lightweight and deployable OCR solution that can be used in resource constrained settings, such as community driven projects, edge deployments and institutions with limited infrastructure.

The thesis work yielded two conference submissions. The full scientific article, titled **Lightweight OCR in North Sámi Heritage Digitization for Smart Learning**, in IEEE format, was *accepted* to the International Conference on Information, Intelligence, Systems and Applications (IISA) 2026 (theme: *21st century AI and education*) at the University of the Aegean, Rhodes, Greece. The article addresses the research question *Can a lightweight OCR architecture approach Transformer level accuracy for North Sámi while enabling practical deployment on modest hardware?*. An extended abstract of this work, titled **OCR and Machine Translation Pipeline for Low Resource Sámi Languages**, was *accepted* to the 15th Scandinavian Conference on Artificial Intelligence (SCAI) 2026 (theme: *AI for Good: Innovation, Ethics, Society*) at the University of Southern Denmark (SDU), Odense, Denmark, and will be presented on June 15th during the poster session.

All the technical details, experiments and scientific claims are covered in the IISA paper. Some sections, such as related work only exist in the scientific article, but some sections are duplicated or expanded on in this report for fuller picture. This report focuses on the *process*: how the project evolved, what was achieved, what was abandoned, and reflections on what worked, what did not, and lessons learned.

1.2 Sámi Languages

The Sámi are the indigenous people of the Sápmi region (Figure 1.1), which spans the northern parts of Norway, Sweden, Finland and Russia, with archaeological evidence of habitation in the region for several thousand years [1]. From the 17th century onward, Swedish, Danish Norwegian and Russian state

expansion brought taxation, Christian missionisation and the suppression of pre-Christian religious practice [1, 2]. In the 19th and 20th centuries this escalated into explicit acts of assimilation: Norwegianization in Norway [3], the *nomadskola* system in Sweden [4], boarding schools in Finland [5], and Soviet collectivisation of the Kola Sámi [6]. After the Alta controversy (1979 to 1981), the protests against the construction of a hydroelectric power plant on the Alta River, more rights were granted to the Sámi people [3]. Sámi parliaments started to operate in all three Nordic countries (Finland’s body dates to 1973) [7], the Sámi flag was adopted in 1986, and Norway formally apologised for the assimilation policies in 2024 [8, 9]. However, the legacy of suppression has left the Sámi languages underrepresented in digital resources, a significant barrier to heritage digitisation and language revitalisation efforts [10].

There are nine mutually unintelligible spoken Sámi varieties across the Sápmi region [12]. The most widely spoken variety, and the focus of this project given the data and tooling available, is North Sámi (ISO 639-3 *sme*, *Davvisámegiella*), with around 25,000 speakers, while the most endangered varieties have fewer than 50 speakers, and some are no longer actively spoken [13, 12]. The Sámi languages belong to the Uralic family, which also includes Finnish, Estonian and Hungarian [12]. According to UNESCO’s *Atlas of the World’s Languages in Danger* [13], North Sámi is classified as *definitely endangered* (children no longer learn the language as a mother tongue in the home), while the more endangered varieties (Pite, Ter, Ume) are classified as *critically endangered* (the youngest speakers are grandparents and older, who use the language partially and infrequently).

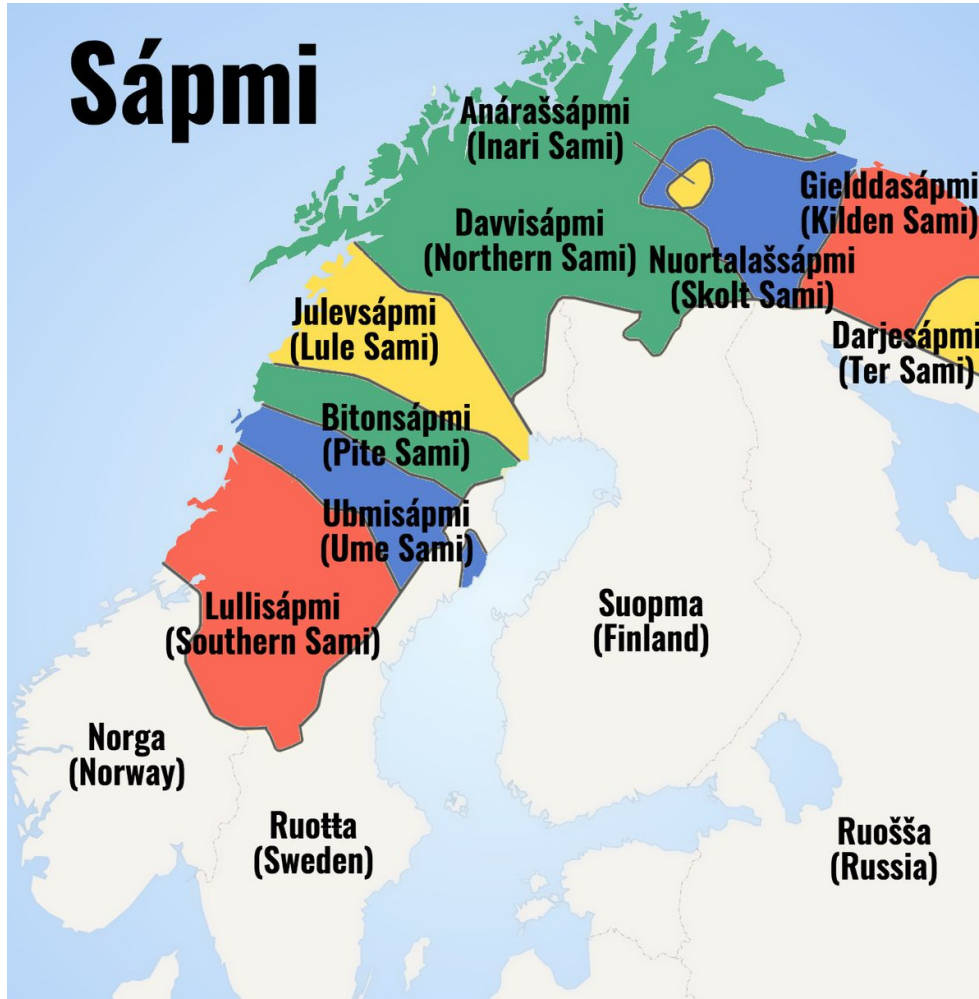


Figure 1.1: The Sápmi region, spanning the northern parts of Norway, Sweden, Finland and Russia, with the geographic distribution of the nine spoken Sámi varieties. North Sámi (*Davvisápmi*) covers the largest area. Source: The Decolonial Atlas [11], reused under the Decolonial Media License 0.1.

1.3 Linguistic Characteristics of North Sámi

North Sámi is an agglutinative¹ language with complex morphology and hundreds of inflected² forms per verb [14]. This agglutinative morphology yields

¹ Agglutinative languages build words by stacking distinct morphemes onto a stem, with each suffix typically carrying a single grammatical meaning. For example, North Sámi *viesuiguin* “with the houses” decomposes as *viesu-* (‘house’) + *-i-* (plural) + *-guin* (‘with’, the comitative case).

² Inflection refers to systematic modifications of a word that mark grammatical categories such as case, number, tense, or person. North Sámi nouns inflect for six cases in singular and plural, while pronouns and verbs additionally distinguish a dual number, yielding many distinct surface forms per stem.

large type token ratios even in modest corpora: meaning that the OCR model rarely sees the same exact word twice during training, so it cannot memorise word shapes altogether and is forced to learn the script at the character level. The orthography of North Sámi further complicates this by including seven special characters (diacritics and modified letters: á, č, đ, ŋ, š, ʦ, ž) that distinguish it from the standard Latin script [14].

It also possesses a three way quantity contrast. There are three consonant lengths (short, long, and overlong), and the difference between these lengths carries a grammatical meaning. In writing, only a part of this is captured: long and overlong consonants are both spelled the same way, as a doubled letter, and the two are told apart only in specialized linguistic notation [14].

The length difference, in North Sámi, between single and doubled consonants, does grammatical work through a process called consonant gradation. For example, the noun *giella* “language” is spelled with a double *ll*, but in a possessive form, *giela* has only a single *l* [14]. In other words, that single letter is the only thing marking the difference between the two grammatical forms. If an OCR system drops or misreads one character, it can change the grammatical meaning of the word entirely. This is why getting individual characters right matters so much for the NLP tasks that follow.

Before 1979, the three Nordic countries used separate standardised orthographies for North Sámi, and a unified orthography was only adopted that year (last revised in 1985) [14]. Many historical documents are therefore written in previous conventions, adding a further layer of complexity for OCR models, which must generalise across orthographic registers to be effective for heritage digitisation. For example, Turi’s *Muitalus Samid Birra* (1910)[15]³ uses the Friis orthography. “An Account of Sami” (eng), is the first secular book released in North Sámi, tells about the life of people herding reindeer in the Jukkasjärvi region of northern Sweden at the beginning of the 20th century, and is included in the out of distribution evaluation set to test the model’s ability to generalise across orthographic conventions.

okta lea dáhpáhus go bodii dárrolaš njuorjovuopmegierragii

It once happened that a Norwegian settler came to the uplands

Á/á áhku (grandmother)	Č/č čáhci (water)	Đ/đ odda (nose)	Ŋ/ŋ eaggals (crystals)	Š/š šaldi (bridge)	ʦ/ʦ máŋolaš (possible)	Ž/ž iežá (oliver)
------------------------------	-------------------------	-----------------------	------------------------------	--------------------------	------------------------------	-------------------------

Seven diacritical characters distinguish Sámi from standard Latin alphabet

Figure 1.2: The seven special diacritical characters that distinguish North Sámi orthography from the standard Latin script. For each character, an example word is shown that uses it. The visual similarity between several pairs (e.g. č/c, š/s, ž/z, ʦ/t) makes their reliable recognition one of the central technical challenges for an OCR model trained on this script.

³In standard modern North Sámi orthography this title is written *Muitalus sámiid birra* (the form used elsewhere in this report). The spelling here preserves Turi’s original 1910 Friis orthography to illustrate the early conventions.

1.4 Personal Motivation

I chose this topic because I find it personally meaningful to work on a project that has the potential to contribute to the preservation of endangered languages. I think that the democratization of technology is an important matter in modern life. Even in resource constrained settings, the same tools should be available to everyone, and that includes the tools for digitizing and preserving cultural heritage. Additionally, I was intrigued by the technical challenges in OCR for low resource languages, and I wanted to explore how machine learning techniques could be adapted to work effectively in such contexts. Lastly, I come from a region situated in southern Poland, where the Silesian language is spoken, which is also a low resource language. This personal connection further motivated me to contribute to this field of research.

1.5 Technical Context

The project uses Python 3.11. The whole development environment, system libraries and CUDA driver are declared in a single `flake.nix` using Nix flakes⁴. Entering the shell with `nix develop` provides an identical environment on both the local development laptop and the uCloud GPU node, with Python dependencies pinned via `requirements.txt` (captured by `pip freeze` inside the flake shell for full reproducibility).

The machine learning stack is built on PyTorch 2.11 [16] with `torchvision` for the ImageNet pretrained backbones and CUDA 13 with `cuDNN` for GPU acceleration. The Hugging Face ecosystem provides the supporting infrastructure: `datasets` [17] streams the Språkbanken North Sámi OCR corpus directly from the Hub, and `safetensors` handles model checkpoint serialisation.

Image preprocessing relies on `opencv-python` and `pillow`, while the `pdfminer` and `NLTK` power the PDF parsing and sentence segmentation used by the alignment TUI tool.

Evaluation uses community standard implementations: `jiwer` for character and word error rates (CER, WER) on the OCR side, and `sacrebleu` [18] for BLEU on the translation side, ensuring the reported scores follow the same definitions used in the literature. Supporting analysis and visualisation is done with `numpy`, `pandas`, `scikit-learn`, `matplotlib`, `seaborn` and `wordcloud`, with exploratory work hosted in Jupyter notebooks.

Compute is run on NVIDIA Tesla V100 32GB on uCloud (SDU/DeiC) for training and a development AMD Ryzen 5 PRO laptop CPU for the inference benchmark, purposefully chosen to reflect a realistic low resource deployment environment rather than ideal serving hardware.

⁴Nix flakes, <https://nix.dev/concepts/flakes>.

2 Goals

While there are some rule based systems for North Sámi in place, such as those provided by Giellatekno [19], including morphological analysers, dictionaries, grammar parsers and single language corpora. In other digitalisation sectors the Sámi languages remain underrepresented, which is itself a barrier to heritage digitisation and language revitalisation efforts [10]. The most accurate publicly available OCR models for North Sámi are transformer based variants fine tuned by the National Library of Norway on synthetic data and released through the Språkbanken Sámi OCR collection [10, 20]. However, these models are computationally heavy. They take 12+ seconds per image on a CPU and around 350 MB of storage memory, which makes them impractical for community driven projects, mobile field documentation, and institutions with limited infrastructure. As shown in Figure 2.1, there is a clear gap in the design space for models that balance accuracy with deployability, particularly for low resource languages like North Sámi.

This project aims to explore whether a lightweight OCR can achieve a high accuracy for North Sámi while allowing practical deployability. Additionally, the project tests the out of domain robustness of such lightweight models on historical lexical content, which is crucial for heritage digitisation, and verifies its feasibility as a component in an end to end OCR→MT pipeline for

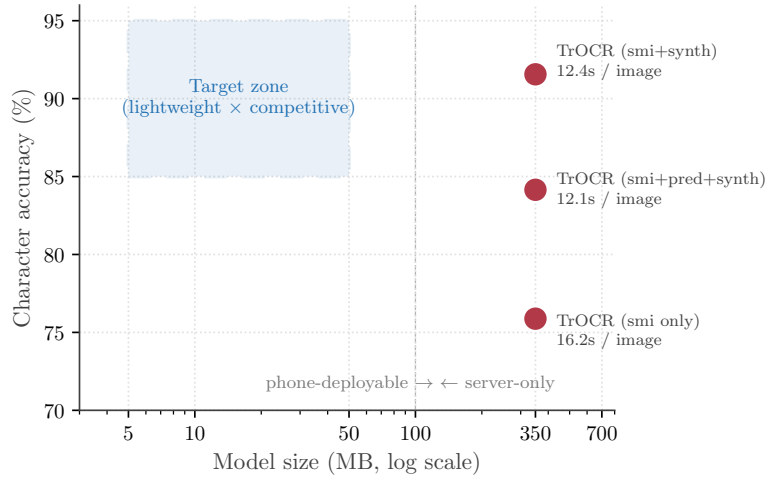


Figure 2.1: Design space of North Sámi OCR models, plotting accuracy against model size (MB) and inference time (seconds). Existing transformer based models cluster in the high accuracy but large size region, while the proposed lightweight CNN-CTC model aims for a target zone of competitive accuracy with significantly reduced size and inference time.

smart learning and archival indexing use cases. After analysing the benchmarking results, further conjunction based sentence splitting is proposed as a linguistically motivated preprocessing step to mitigate the fixed width input penalty of the lightweight architecture, showing that architectural and linguistic adaptations can work together to improve performance under deployment constraints.

2.1 Research Question

The main research question guiding this project is:

Can a lightweight OCR architecture approach Transformer level accuracy for North Sámi while enabling practical deployment on modest hardware?.

The emphasis on deployment constraints is a crucial aspect of the project, as it looks into one of the often overlooked parts of research: the practicality and usability of the proposed solutions. The project doesn't focus solely on accuracy, but also looks at ways of mitigating some technological constraints given by the hardware and resource limitations.

2.2 Specific Goals

As listed in the scientific article, the six main contributions of this project are:

1. A lightweight CNN-CTC architecture for North Sámi OCR, achieving a competitive balance of accuracy and deployability.
2. A systematic benchmark comparing the proposed model against 7 TrOCR variants and 3 CTC variants with ImageNet pretrained backbones, demonstrating the tradeoffs between accuracy and computational efficiency.
3. A contribution to Sustainable AI through a lightweight CNN-CTC model.
4. A linguistically motivated conjunction based sentence splitting method that mitigates the fixed width input penalty of lightweight architectures, improving long sentence accuracy under oracle ground truth conditions, as a proof of concept.
5. An end to end OCR→MT pipeline integrating the proposed model with TartuNLP's North Sámi English API, demonstrating usability for downstream tasks.
6. Demonstrating usefulness in 21st century education, AI & NLP, and next generation robotic systems applied to low resource languages.

3 Process

Readers without an ML/AI background may find the brief explanation of concepts from this chapter in Appendix A.

3.1 Dataset Exploration

First, the Språkbanken Sámi OCR corpus [21, 10] from Hugging Face was analysed. It consists of train/validation/test splits (307k/40.8k/84.5k rows), meant for fitting OCR models for North Sámi, Lule and Inari Sámi. Each sample is a single synthetically rendered text line paired with its ground truth transcription, e.g. those shown in Figure 3.1. The line by line format meant that line detection and layout analysis were outside the scope of the model. The OCR goal was to transcribe the line of text into a string. Extracted vocabulary yielded a 395 symbol charset (394 characters plus one CTC blank token) later used by the CTC head. The computed word and letter frequencies are rendered as the clouds in Figure 3.2. Two observations from this step shaped later decisions. The conjunction *ja* (“and”) dominates the word distribution, foreshadowing the conjunction splitting preprocessing. The seven special characters appear at non-negligible frequencies.

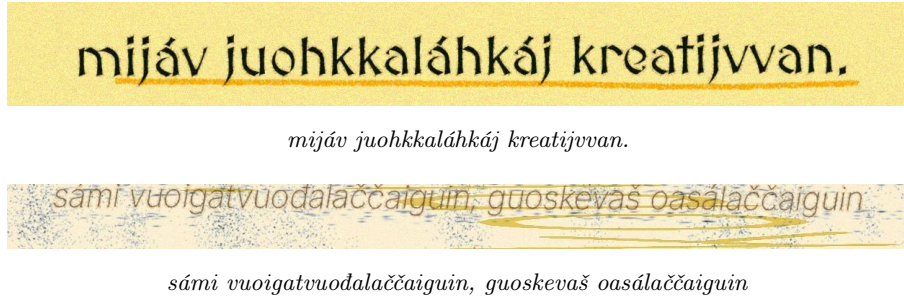


Figure 3.1: Two Språkbanken synthetic samples shown as the model sees them. The rendering varies font, spacing, slight noise and background to simulate scanned text appearance.



3.2 Corpus Building Detour

A small detour was taken during the early stages of the project to assess whether a parallel English Sámi corpus could be built from Turi’s *Muitalus sámiiid birra* (1910). The idea was to explore a possibility of training a MT solution using the yielded corpora (e.g. Gated Recurrent Unit (GRU) Network). Both versions of the book were obtained as PDFs, one in North Sámi (Friis’ orthography), the other in English (translated and edited by Thomas A. DuBois), alongside an already parsed, sentence split Sámi version from the Gielletekno website.

The PDFs are parsed with `pdfminer`, split into sentences with `NLTK`, and aligned with a custom TUI tool that supports manual approval (See Figure 3.3), deletion, and merging of sentence pairs. Due to time constraints, the tool was ultimately not used to generate a full bilingual corpus, instead, it produced a small evaluation set of 48 synthetic renderings of the Sámi sentences, which became the out of domain test set in the IISA paper.

NORTH SAMI		ENGLISH	
0	Mon lean okta sápmelaš .	0	I am a Sámi
1	Mon lean okta sápmelaš . guhte lean bargan viht sámi bargguid .	1	I am a Sámi who has done all sorts of Sámi work and I know all
2	Ja mon lean ipmirdan , ahte Ruođa hállehus háláida min veahkeih .	2	I have come to understand that the Swedish government wants to
3	Ja dassa lea dat siivva : go sápmelaš bohta moskkus gámmrii , d	3	And this is the reason : when a Sámi becomes closed up in a ro
4	Su jurgadat eal golgga , go leat seináint ja moskkus oaivvi nal	4	His thoughts don't flow because there are walls and his mind is
5	Ja ii leatge buorre suttje orrut suhkkus vuovdiid siste , gos l	5	And it is also not good at all for him to live in dense forest
6	Muhto go sápmelaš lea alá váriid nalde , de sus lea oah čielg	6	But when a Sámi is on the high mountains , then he has quite a
7	Ja jos doppe luvččui oktaKámbáidni soames alá vári nalde , de v	7	And if there were a meeting place on some high mountain , then
8	Mon lean jurgadán , ahte dat luvččui buoremus , jos luvččui da	8	I have been thinking that it would be best if there were a book
9	Ja dassa lea sápmelaš buorre suttje orrut suhkkus , vai boa	9	And there would be good to write about all the events and furnish a
10	Ja lea dat nuoppeida sápmelaš hávski guhtat sámi dili birra .	10	And it would be nice also for other Sámi to hear about Sámi con
11	Dás guhtá eanemus Juhkásgávdni suohkana sámiid birra .	11	Most of what is described here has to do with Sámi in the Jukka
12	Boarráseamos mitalusat sámiid birra , dat maid nuohpi sápmi lea	12	The Oldest Stories of the Sámi Such As Told From One Sámi To An
13	Dás leat juohkealagáš mitalusat , muhto ii leat vissis leatgo j	13	Herein are all sorts of stories , but it is not certain whether
14	Mitalusat sápmelašcaid birra , gos sii leat vuohččan orron , ja	14	A Story About The Sámi , Where They Used To Live , And How Thei
15	Sámiid boahhtimusas ii leat gullon , ahte luvččui boahččan gosge	15	The Sámi have always lived in these parts , here in the Sámi ho
16	Sápmelaš lea leamaš dáppeorru miehta Sámi eatnama .	16	And in those days when the Sámi were living up by the coast , t
17	Ja go sápmelaš lea leamaš doppe mearragáttiin orrum , dalle ea	17	And on the Swedish side of the border , Sámi also lived everywh
18	Ja sámit leat orron Ruođa bealde maidde juohke báikkis .	18	And back then there weren't any settlers at all ; the Sámi didn
19	Eai deatna ahte sii leat eallán livvudn vuohččan , guhtii j	19	I think that they lived at first off fish , birds , and wild me
20	Mon jurgadán ahte sii leat eallán livvudn vuohččan , guhtii j	20	This is why I believe so : one finds Sámi men for netting pla
21	Man dihte mon doavivun dat : go gávdnojit sámegeilat namat várp	21	And an even greater indicator that Sámi have lived for a long t
22	Ja dat lea velá stuorit mearka , ahte sámit leat orron guhká o	22	These pits were their godpit .
23	Dat leat dat rokkit mat leat eatnama siste juohke sajis , gos d	23	And they dug burrows like other animals of the wilds .
24	Dat rokkit leat leamaš sin goadiit .	24	And most of these pits one finds along the coast and on lakesid
25	Ja dahke sii rokkiiid goe ležánna meahieallit .	25	Some of these pits were meat caches and some were pits for capt
26	Ja eanemus oassi dain rokkiiin gávdno mearragáttiin ja jávre- ja	26	These latter were deep and had a covering over them , so that w
27	Soamis dein rokkiiin lea leamaš biergosuodjin ja soamis lea leam	27	And at the bottom of the pit there were spikes of wood and antl
28	Dat lea čienat ja lohkiid nalde , ja go eallit bodit nala , de j	28	People don't think that these pits were made by people or were
29	Ja dan rokki vuodus leađs sákkit muoras ja soarvvis , ja dan na	29	And indeed old Sámi have noticed that these pits were also used
30	Ja dat oah lea oahččan ahte sámiid leat leamaš oahnu	30	And he has happened on a grave pit that a Sámi has slept
31	Ja leat geat boares sámit ožžon mearkka , ahte dat leat hávdit d	31	And its face was like lichen
32	Dat lea nu dahhuvuvvan mánnga háve , ahte sápmelaš lea nohkkan	32	And it said : "Do not sleep upon me ! " And if he did not compl

Figure 3.3: Alignment CLI tool for parallel corpora generation from two sources.

3.3 Training Pipeline Overview

The modular OCR pipeline consists of three slots: a *backbone*, an *encoder* and a *decoder*. Each slot is easily swappable, yielding 15 distinct configurations in total: 5 backbones (SimpleCNN, VGG16, VGG19, ResNet50, ResNet101) $\times$ 3 encoders (none, BiLSTM, Transformer), with a fixed CTC decoder across all variants. A training queue runs configurations consecutively without manual intervention, logging each step and storing checkpoints and outputs in a structured way for monitoring and debugging. The modular design also leaves room for future extensions, such as a post correction module or a scan quality classifier.

This layout is inspired by Shi et al.’s CRNN [22]: a CNN backbone extracts spatial features, a sequence encoder adds temporal context, and a CTC head maps the encoded sequence to characters. Each component is explained in more detail below.

Backbone

The backbone slot contains the CNN feature extractor, which processes the input image into a sequence of column features. The proposed lightweight variant is a 7 layer SimpleCNN trained from scratch, while the comparison variants use ImageNet pretrained VGG16, VGG19, ResNet50 and ResNet101 backbones with differential learning rates to preserve pretrained features.

Encoder

The encoder slot contains the sequence modelling component, which captures contextual dependencies between characters. The options are a 2 layer BiLSTM, a Transformer encoder block, or no encoder (direct CNN to CTC). This allowed to isolate the contribution of explicit sequence modelling on top of the per column CNN features.

Decoder

The decoder slot contains the transcription head, which maps the encoded features to character probabilities. A CTC decoder [23] is used across all configurations, which allows for training without explicit character level alignment and handles variable length outputs. This fixed decoder setup ensures that differences in performance can be attributed to the backbone and encoder choices.

3.4 Comparison

The modular pipeline allowed for a systematic comparison of the CNN based variants against the transformer based models. The comparison benchmark

was run on CPU to reflect realistic deployment conditions, and evaluated on the same test set using consistent metrics (CER, WER, exact match accuracy).

Due to limited accessibility to uCloud resources only the following variants were trained: `ctc_simple` (the lightweight 7 layer SimpleCNN trained from scratch), `ctc_vgg16` and `ctc_vgg19` (ImageNet pretrained VGG backbones with differential learning rates), and `ctc_resnet50` (ImageNet pretrained ResNet50). The `ctc_resnet101` variant was gated on the ResNet50 results. Per epoch time on the V100 was ~ 19 minutes for the VGG pair and ~ 76 minutes for ResNet50, so a full 100 epoch run would have occupied the GPU for ~ 32 hours (VGG) or ~ 5 days (ResNet50). In practice all three pretrained variants triggered early stopping well before that: `ctc_vgg16` at 52 epochs, `ctc_vgg19` at 63 epochs, and `ctc_resnet50` at 37 epochs (~ 48 hours), with validation CER stuck around 0.69 to 0.73 at early stopping (ResNet50’s final test CER of 0.753 reflects continued degradation past that point, see Section 4.1). The compute budget made it impossible to commit further.

3.5 The lightweight CNN-CTC variant

The headline lightweight variant, `ctc_simple`, uses a 7 layer SimpleCNN backbone trained from scratch, no encoder and a CTC head over the 395 symbol vocabulary. The architecture is shown in Figure 3.4.

The SimpleCNN has 7 convolutional layers with following channel widths 64, 128, 256, 256, 512, 512, 512, each followed by batch normalisation and a ReLU activation function. Pooling is asymmetric: 2×2 max pooling in the early layers progressively collapses both axes, while the later layers switch to 2×1 pooling that keeps horizontal resolution intact, preserving the character ordering that the CTC head relies on. A final adaptive pool reduces feature map height to 1, yielding a sequence of column vectors. These are projected to 256 dimensions and passed directly to a linear head that maps to 395 classes (394 characters plus the CTC blank) followed by LogSoftmax. The full model has ~ 6 M parameters. It is trained with a CTC loss that marginalises over all valid character alignments and decoded greedily at inference time. In practice it ran the full 100 epoch schedule at ~ 8 minutes per epoch (~ 13 hours).

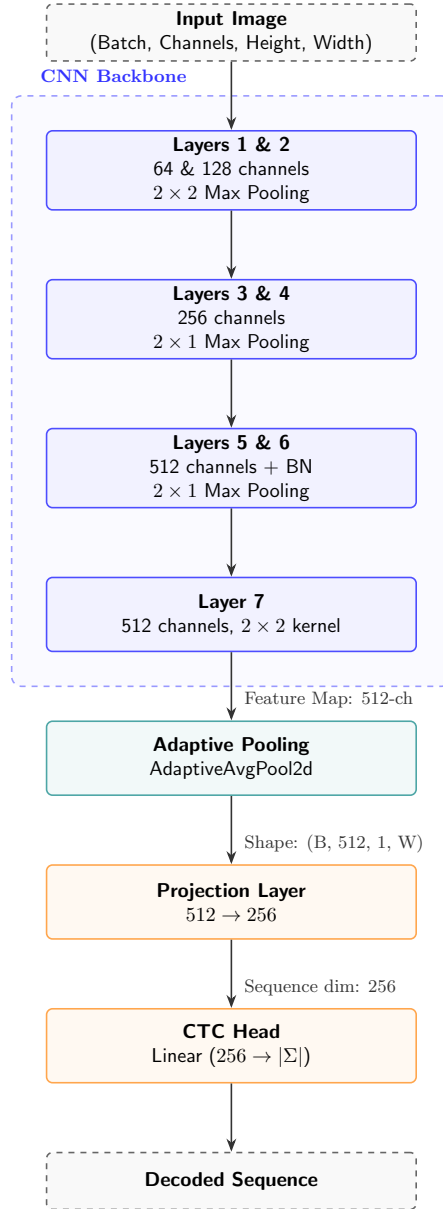


Figure 3.4: Modular OCR pipeline used in the IISA paper. The lightweight `ctc_simple` variant uses a 7 layer SimpleCNN backbone trained from scratch and a CTC head over the 395 symbol vocabulary, the encoder slot is left empty.

3.6 Training Setup

All CTC variants share the same training recipe with backbone specific overrides. The optimiser is AdamW with a base learning rate of 10^{-4} and weight decay 10^{-4} for `ctc_simple`, reduced to 3×10^{-5} with a $0.1\times$ differential multiplier on pretrained VGG and ResNet50 layers. The batch size is 32 for SimpleCNN and VGG, reduced to 16 for ResNet50 due to GPU memory. Training runs for up to 100 epochs with early stopping (patience 15 to 35 epochs, scaling with model size), `ReduceLROnPlateau` halves the learning rate when validation CER plateaus, and gradients are clipped to a max norm of 5.0 to stabilise the CTC loss.

The training data is the Språkbanken synthetic North Sámi OCR corpus [21, 10], with 276,000 samples in the training subset (90% of the 307,387-sample training split) and roughly 31,000 held out for development.

3.7 Evaluation and End to End Pipeline

Evaluation is conducted on a 1,048 sample combined test set: 1,000 held out samples from the Språkbanken validation split (in domain) and 48 synthetic renderings of Turi’s 1910 *Muitalus sámiiid birra* (out of domain lexical content). Models are scored with character error rate (CER), word error rate (WER), exact match accuracy, and a normalised accuracy variant that removes punctuation and extra whitespaces, all computed as Levenshtein distance on raw UTF-8 strings. Per image inference time is measured on CPU to reflect realistic deployment conditions. The TrOCR variants are used as released by the National Library of Norway without additional fine tuning, while `ctc_simple` is trained from scratch on the Språkbanken corpus. The benchmark compares a ready pretrained baseline against a purpose trained lightweight model.

The full end to end pipeline passes the OCR predictions into the TartuNLP North Sámi English machine translation API [24], with the translated output scored against the English reference using BLEU, chrF [25], and TER (See Figure 3.6).

3.8 Conjunction Splitting Experiment

The experiment surfaced from the benchmark results. Per sentence error analysis on both `ctc_simple` and the strongest TrOCR variants showed a steep accuracy drop beyond roughly 80 characters, caused by the fixed 800 px input width cutoff. Referencing the most frequently used words in the dataset (Figure 3.5) where the conjunction *ja* dominated the distribution suggested that splitting long inputs at North Sámi conjunction boundaries (*ja* “and”, *muhto* “but”, *dahje* “or”, *go* “when/because”, *ahte* “that”, *joss* “if”) before OCR and rejoin the predictions afterwards, might mitigate that issue.

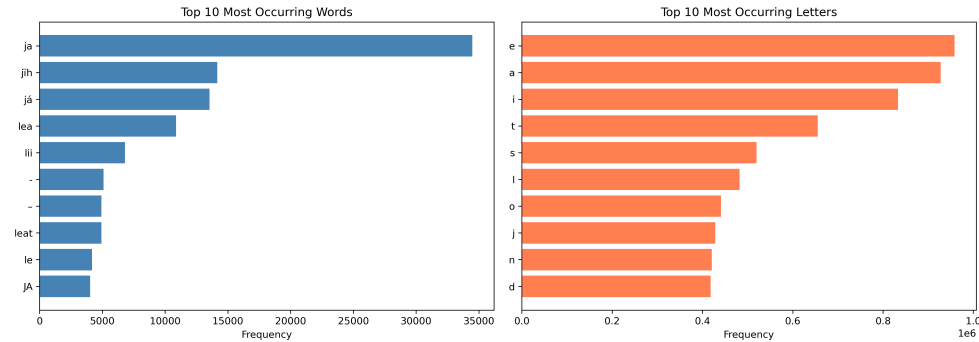


Figure 3.5: Top 10 most frequent words (left) and characters (right) in the Språkbanken North Sámi training corpus. The conjunction *ja* (“and”) is by far the most frequent token, motivating conjunction based segmentation.

More specifically, every input sentence longer than 80 characters is scanned for these conjunction tokens by a case insensitive word boundary regex matching (the implementation also extends to multi-word and subordinate variants such as *ovdal go* “before” and *dasgo* “because”), with the multi-word forms being matched greedily before their single-word counterparts to avoid double counting. The text is split immediately *before* each matched conjunction so the connective stays attached to its following clause, and a minimum segment length of 20 characters guards against over splitting when there are many conjunctions close to eachother. Each resulting text segment is then re-rendered as a fresh synthetic line image through the same generator used for the evaluation out-of-domain data, passed independently through `ctc_simple`, and the per segment predictions are joined with single space to form the final transcription. The benchmark compares per sample CER on the long sentence subset of the test set before and after splitting, with statistical significance estimated via a paired test and effect size reported as Cohen’s *d*.

```
python src/ocr_translate_pipeline.py

≡ Available Models ≡

TrOCR Models (pre-trained):
  1. trocr_smi_nor_pred_synth
  2. trocr_smi_pred_synth
  3. trocr_smi_synth
  4. trocr_smi_nor_pred
  5. trocr_smi_pred
  6. trocr_smi_nor
  7. trocr_smi

Trained Models:
  8. ctc_simple (2026-03-27_queue_1)
  9. ctc_resnet50 (2026-03-28_queue)
 10. ctc_vgg16 (2026-03-28_queue)
 11. ctc_vgg19 (2026-03-28_queue)

Select model (1-11): 8

≡ OCR + Translation Benchmark ≡
Model: ctc_simple
Weights: trained_models/2026-03-27_queue_1/ctc_simple/checkpoint_best.pt
Dataset: test_data/account_of_sami/synthetic_images/
Total Images: 48

[1/3] Loading model...
[2/3] Processing 48 images and translations...
Progress: [> ] 1/48 (2.1%)ID: 1
Image: test_data/account_of_sami/synthetic_images/entry_1.png
GT Sami: mon lean okta sápmelaš
Pred Sami: mon lean okta sápmelaš
CER: 0.0000, WER: 0.0000, Accuracy: 1.00
GT English: i am a sámi
Pred English: i am a saami

Progress: [-> ] 2/48 (4.2%)ID: 100
```

Figure 3.6: CLI tool for running the end to end OCR→translation pipeline. The user selects the OCR model and dataset, and the tool reports both OCR accuracy metrics (CER, WER) and translation quality metrics (BLEU, chrF, TER) on the translated output.

4 Process Reflections

4.1 Main Results

The headline benchmark from the scientific article compares the proposed `ctc_simple` against the National Library of Norway’s best TrOCR variants. The strongest TrOCR variant (`trocr_smi_synth`) sets the SOTA at 8.55% CER (91.56% character accuracy¹), with `ctc_simple` within 3.69 pp at 12.24% CER (87.72% character accuracy). On efficiency the picture flips. The lightweight model runs roughly 380× faster on the same CPU and is 15× smaller (23 MB vs. 350 MB), which is decisive for heritage digitisation. At 0.033 s per image a CPU pushes about 109,000 images per hour, so a million page corpus runs in under 10 hours where TrOCR would need around 144 days, and the small footprint fits edge devices that TrOCR cannot.

Table 4.1 reports the complete benchmark across all 11 models: the seven TrOCR variants released by the National Library of Norway, the proposed `ctc_simple`, and the three ImageNet-pretrained CTC backbones that failed to converge. Extended comparison plot from the scientific article in Figure 4.3.

Table 4.1: OCR benchmark on 1,048 test samples: all 7 TrOCR variants from the National Library of Norway, the proposed lightweight `ctc_simple`, and the three ImageNet-pretrained CTC backbones that failed to converge. Bold marks the best value in each column. Lower CER/WER is better, higher Exact Match Accuracy is better.

Model	Training Data	CER↓ (%)	WER↓ (%)	Exact↑ (%)	sec / img	Size (MB)
<code>trocr_smi_synth</code>	Smi+Synth	8.55	28.16	42.08	12.41	~350
<code>ctc_simple</code>	Smi+Synth	12.24	37.35	36.74	0.033	23
<code>trocr_smi_pred_synth</code>	Smi+Pred+Synth	15.85	45.76	24.14	12.08	~350
<code>trocr_smi_nor_pred_synth</code>	Smi+Nor+Pred+Synth	17.46	48.20	23.38	13.55	~350
<code>trocr_smi</code>	Smi only	24.12	63.50	12.88	16.23	~350
<code>trocr_smi_pred</code>	Smi+Pred	24.82	63.06	16.32	11.730	~350
<code>trocr_smi_nor</code>	Smi+Nor	25.84	66.62	13.17	12.890	~350
<code>trocr_smi_nor_pred</code>	Smi+Nor+Pred	26.43	64.21	17.08	11.759	~350
<code>ctc_vgg19</code>	Smi+Synth	71.86	97.89	0.00	0.055	~550
<code>ctc_vgg16</code>	Smi+Synth	73.94	98.81	0.00	0.044	~550
<code>ctc_resnet50</code>	Smi+Synth	75.30	98.64	0.00	0.034	~100

*Training-data tags: smi=Sámi data, nor+=Norwegian, pred+=auto-transcribed, synth+=synthetic pre-training.

The three CTC variants with ImageNet pretrained backbones (`ctc_vgg16`, `ctc_vgg19`, `ctc_resnet50`) all failed to converge to usable performance (>69% CER, ~0% exact match accuracy) despite sharing the projection layer

¹Character accuracy is measured directly and does not equal 100 – CER, since CER counts insertions as well as substitutions and deletions.

and CTC head with the working `ctc_simple` model. None of the four variants uses a recurrent encoder, so the backbone is the only difference. Two distinct training signatures emerged (Figure 4.1). The VGG pair underfits on both ends, training loss drops from 0.81 only to ~ 0.60 over around 60 epochs and validation CER never breaks the 0.72 barrier. ResNet50, conversely, fits the training set aggressively, train loss collapses from 0.80 to 0.23 in 37 epochs while validation CER stays unchanged at 0.73 and validation loss bounces upward (a likely sign of overfitting). The same underlying outcome appears in both: the pretrained features exposed to the CTC head did not generalise to character level prediction on this dataset.

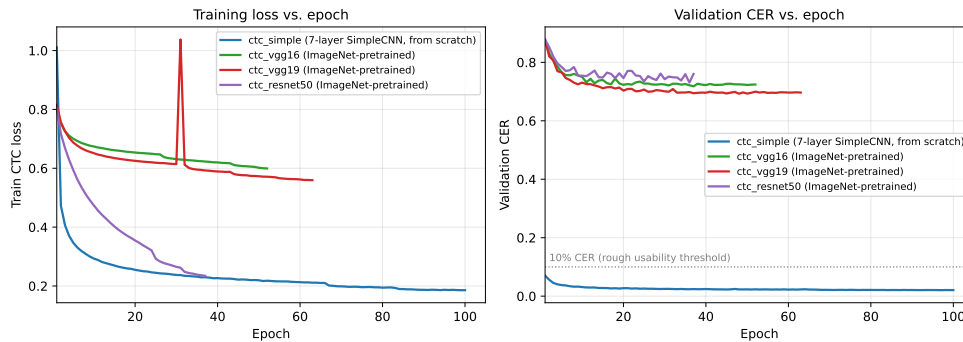


Figure 4.1: Training dynamics of `ctc_simple` (blue) vs. the ImageNet pretrained CTC variants `ctc_vgg16` (green), `ctc_vgg19` (red) and `ctc_resnet50` (purple). Left: training CTC loss, `ctc_simple` converges to near zero, the VGG pair underfits, ResNet50 overfits. Right: validation CER, `ctc_simple` reaches below 10% while all pretrained variants stay stuck around 0.70 until early stopping.

`ReduceLROnPlateau` repeatedly halved the learning rate with no resulting improvement in validation CER for any of the failed variants. This behaviour rules out one obvious reason that learning rate was too high or of gradient instability, because both would normally respond to a smaller step size. The pretrained backbones were already updating at a very low effective learning rate (a base of 3×10^{-5} , further scaled by 0.1 on the backbone to give 3×10^{-6}), so too little learning might have been the issue. The VGG pair illustrates this, with training loss barely moving ($0.81 \rightarrow 0.60$), which is what one would expect if the backbone is learning too slowly to adapt. ResNet50 reveals something else. Training loss falls sharply from 0.80 to 0.23 over 37 epochs while validation CER stays flat. This indicates the head had enough capacity to fit and memorise the pretrained features, but those features did not transfer to the character-level recognition task in the first place.

Concrete prediction examples for `ctc_simple` on the in domain test-set are shown in Figure 4.2. Full results comparison in Figure 4.3.

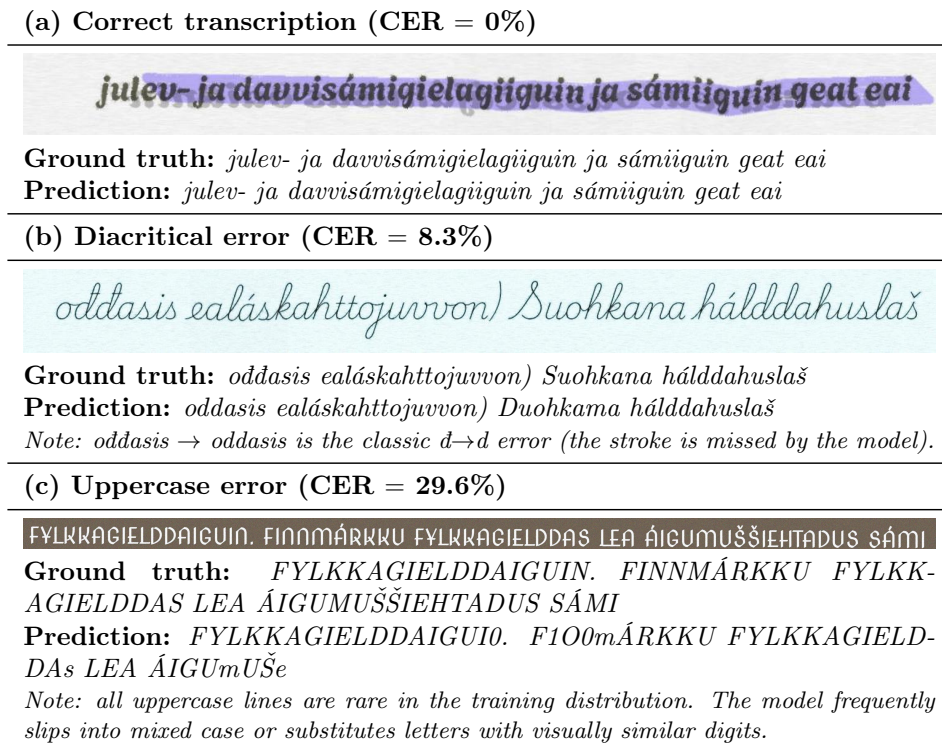


Figure 4.2: CNN-CTC predictions illustrating the model predictions. (a) most lines are accurately transcribed, (b) diacritical strokes are the most common error, (c) all uppercase lines, rare in the training distribution, are by far the worst case.

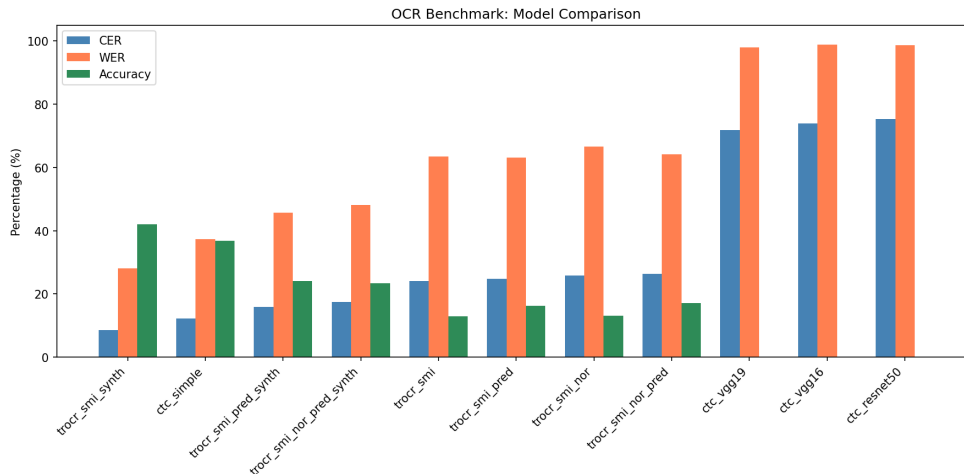


Figure 4.3: Test set CER, WER, and exact match accuracy across 11 models (7 TrOCR variants, the lightweight CNN-CTC, and 3 failed ImageNet pretrained backbones). Lower values are better for CER and WER, higher values are better for exact match accuracy.

4.2 Out of Domain Generalisation

The out of domain evaluation on synthetic renderings of Turi’s 1910 *Muitalus sámiiid birra* produced rather surprising results. In domain, TrOCR has the best accuracy at 8.33% CER on the Språkbanken validation samples alone, against 12.24% for `ctc_simple`.

On the historical text test set, TrOCR degrades by 4.84 pp (8.33% $\rightarrow$ 13.17%) while `ctc_simple` is essentially flat (12.24% $\rightarrow$ 12.21%). One TrOCR variant (`trocr_smi_pred_synth`) still beats `ctc_simple` on raw book CER (11.00%). After punctuation and whitespace normalisation, however, the lightweight model overtakes all TrOCR variants on book text (62.5% normalised accuracy vs. 45.8% for the best TrOCR) (See Figure 4.4).

The sample is small (48 pairs) so it’s nowhere near conclusive, but it might suggest that the smaller model learns more transferable character level features and is less prone to overfitting to synthetic rendering artefacts in the training distribution.

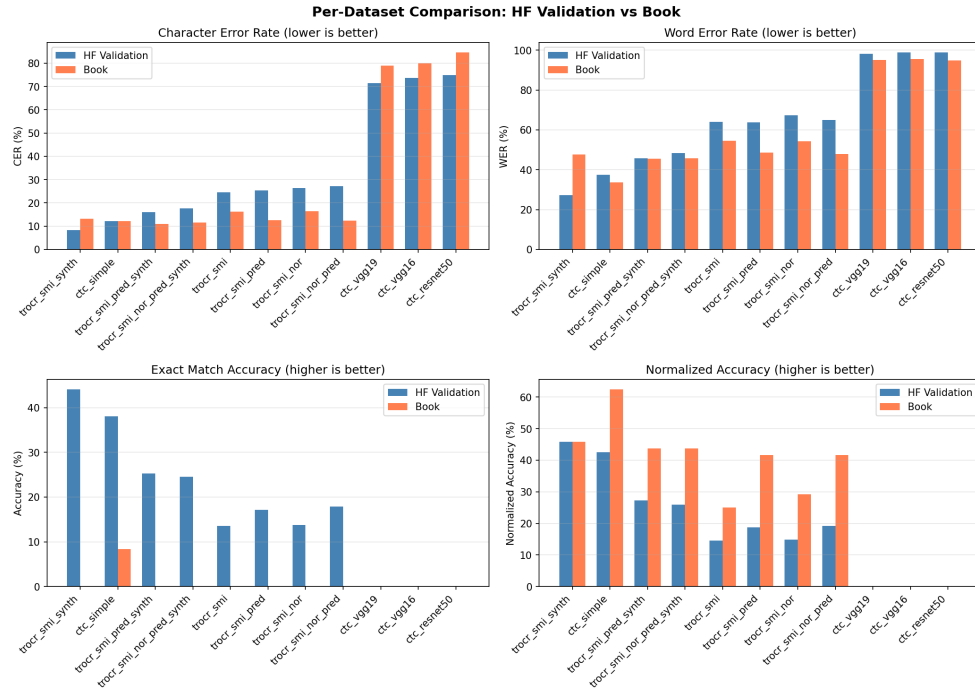


Figure 4.4: Per Dataset Comparison: Språkbanken validation split vs Turi’s 1910 *Muitalus sámiiid birra* benchmark results. Exact match accuracy on the book set is near zero for all models due to punctuation and orthography discrepancies. Normalised accuracy is the more informative metric here.

4.3 Conjunction Based Sentence Splitting

Splitting long inputs at North Sámi conjunction boundaries before OCR and rejoining the predictions afterwards dropped long sentence CER from 15.17% to 8.12%, a 7.05pp improvement, below the short sentence baseline of 12.04% removing the length accuracy degradation (paired test $p = 0.000009$, Cohen’s $d = 1.35$, 95% of split samples improved, none degraded). Sentence length against CER and improvements visible in Figure 4.5.

It is worth mentioning that the splitter acts on the ground truth strings rather than on a input OCR prediction, which makes the reported numbers a best case scenario, for the deployable variant would have to detect conjunctions on a noisy data (where short tokens like *ja* and *go* may themselves be misrecognised) or fall back to whitespace based segmentation of the line image, both of which are expected to lose much of the stated 7.05pp gain. In addition, each segment is re-rendered as a fresh clean synthetic line image before OCR, so the experiment idealises both the segmentation *and* the image quality, further inflating the reported upper bound.

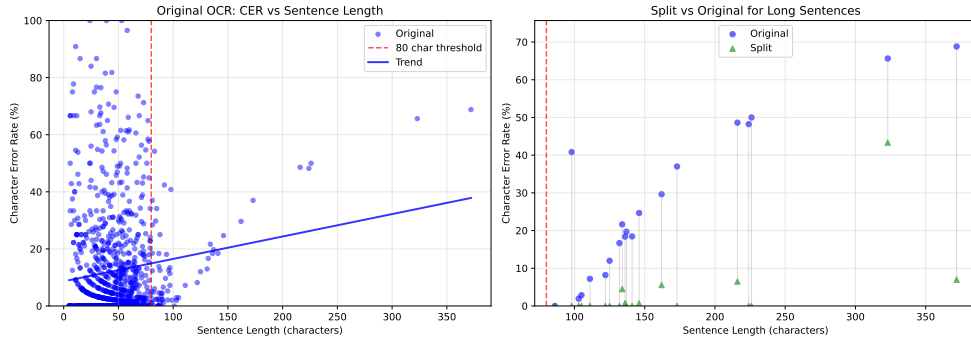


Figure 4.5: Character Error Rate of the lightweight CNN-CTC against the sentence length and the CER change after the split experiment.

4.4 End to End Pipeline

After passing the OCR predictions into TartuNLP’s North Sámi English machine translation API [24]. The output scored BLEU 10.40, chrF 33.85 [25] and TER 76.30% on the out of domain parallel set.

The moderate translation quality reflects how OCR errors propagate downstream, and the effect is even more pronounced for agglutinative languages. At 12.24% CER, many of the relatively minor character errors land on the word endings that signal a noun’s grammatical role or attach meaning like tense, number, and possession, rather than on the core part of the word that carries its meaning. Because these endings encode how words relate to one another in a sentence, a single dropped character can flip way the whole clause is read

and degrade MT accuracy further. The results do not yield a fully fluent automatic translation, but they show that an end to end pipeline is in fact viable, even if for now it is best suited to simple downstream tasks such as archival indexing or content discovery rather than complete translation.

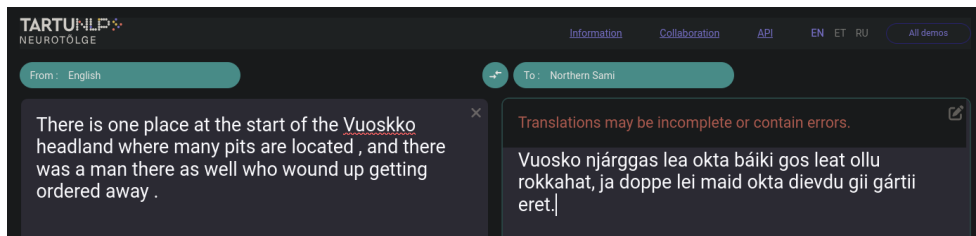


Figure 4.6: The TartuNLP translation web interface, available at <https://translate.ut.ee/>. The same backend is used programmatically in the end to end pipeline via TartuNLP API.

4.5 Applications

The lightweight OCR models allows multiple application cases. The community driven preservation projects (small archives, local museums, language enthusiasts) typically operate without dedicated GPU or many resources at hand. A model that runs on a phone or a small laptop and fully offline, would remove the deployment friction that would otherwise keep these efforts away from being realized. The lightweight model is realistic as the OCR layer in a search and indexing pipeline over million page historical corpora in archival context, or as a back-end for educational tools that need to react to text in real time (for example, mobile flashcards that take a photo of a Sámi book page). Lastly, a combination of the CNN-CTC model with a generative model (a Stable Diffusion inpainting pipeline) for joint recognition and reconstruction of occluded text. Figure 4.7 shows the proof of concept output, an artificially occluded North Sámi line is fed through OCR and inpainting, producing both a transcription and a plausible reconstruction of the original image with damaged text. This kind of pipeline becomes possible only when each component is small and fast enough to run together on an embedded device, useful in self driving settings (partially obscured road signs), in retail (damaged product labels), or in next generation robotic assistants where text in the visual scene must be interpreted continuously.



Figure 4.7: Proof of concept damaged text reconstruction combining `ctc_simple` with a generative inpainting model on an artificially occluded North Sámi line. The pipeline produces both a transcription and a reconstructed image. Enabled by the lightweight footprint, out of reach for the TrOCR baseline. Demonstrated as a future work direction in the IISA paper.

4.6 Future work

The project leaves several questions unanswered and opens multiple options for future work. The main question still open to investigation is robustness to physical degradation. The entire benchmark in this thesis is conducted on synthetic renderings, and collecting and annotating a small corpus of real scanned Sámi documents (with realistic artefacts such as paper texture, ink bleed, skew and historical fonts) would validate the project’s applicability. The CTC model is weakest on rare inflected forms. Integrating Giellatekno’s morphological analyser [26] as a post correction layer, for instance by rescoring the top k CTC hypotheses against a finite state lexicon, could selectively repair exactly the errors that propagate worst into downstream MT, without enlarging the deployed model. Next, as `ctc_simple` works on line level, applying Layout analysis and full page processing by pairing it with a lightweight line and region detector would elevate its usability from transcribing a cropped line to being able to process a full page, which matters for archival use cases. Furthermore, it would be beneficial to check if the same approach works on different languages. Retraining the same architecture on other Uralic varieties (Inari, Skolt, Lule Sámi) or on unrelated low resource scripts would test if the same choices generalise, and if the same ImageNet pretraining issue appears. Finally, because the current decoder is purely greedy, showing the model’s confidence in its predictions would increase the model’s transparency.

4.7 Conclusion

The honest answer to the research question from (Section 2.1) is: **partially**. Dependant on the specific aspect of the question, the answer ranging from yes to inconclusive.

- **Deployment:** The size and speed advantage of `ctc_simple` over the strongest TrOCR baseline is the difference between a million page archive

processable in hours on widespread available hardware and one that would take days, and the difference between an OCR model that fits on a phone and one that does not. Unequivocally addressing the goal as achieved.

- **Accuracy:** On the in domain synthetic benchmark, `ctc_simple` trails the best TrOCR variant by only 3.69 pp in CER, closer than the size difference would predict, and on the out of domain historical text subset it actually overtakes TrOCR on normalised accuracy. However, for an agglutinative language those few percentage points make a huge difference. The downstream OCR→MT pipeline might be enough for archival indexing or content discovery but not for fluent automatic translation. Answering the question as yes but with few caveats.
- **Validation:** The entire evaluation in this thesis is conducted on synthetic renderings, with 48 out of domain samples drawn from a single 1910 book. The model’s behaviour on real scanned documents, with realistic paper, ink and degradation artefacts, has not been tested. As of now, the question remains unanswered.

The six contributions stated in Chapter 2 are all delivered: a lightweight CNN-CTC architecture, a systematic 11 model benchmark, a Sustainable AI footprint argument, an end to end OCR→MT pipeline, a linguistically motivated conjunction splitting method, and a discussion of education, AI & NLP and robotics applications. The three ImageNet pretrained CTC baselines did not converge, so the within CTC comparison is partial, the out of domain set is small, and no real scan data was used.

A Background Concepts and Methods

A.1 Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a deep learning architecture that processes images by sliding small, learned filters (kernels) across the input and stacking the resulting activations into feature maps. The filters are shared across layers, which reduces the parameter count compared to a fully connected network of the same depth. Stacking many convolutional layers lets the network learn a hierarchy of features: simple edges and textures in the early layers, character level structures in the deeper ones.

Feature maps. Output of the convolutional layer. Each convolutional filter produces a feature map, it's a 2D array of activations indicating where in the image the pattern, the filter has learned (edges, corners, or a diacritical marks), is present. The set of input neurons that project to the same convolution neuron is called its *receptive field*. As the network goes deeper, the receptive field grows, allowing later layers to capture larger spatial patterns. In OCR, this means that deeper layers can learn to recognise whole characters or character combinations, while early layers focus on local features like strokes and diacritics.

Activation Function After each convolution, a non linear activation function (ReLU in this case) is applied element wise to the feature maps. This allows the network to learn complex, non linear relationships between the input pixels and the output features. ReLU (Rectified Linear Unit) is a common choice because it mitigates the vanishing gradient problem and promotes sparse activations, which can improve learning efficiency.

Subsampling (Pooling). Pooling layers downsample feature maps, typically by taking the maximum (max pooling) or average activation value over a small spatial window. They make the representation more compact, enlarge the receptive field of subsequent layers, highlight the most important features, and introduce a small amount of translation tolerance.

Column features (adaptive pooling). For OCR, the 2D feature maps produced by the CNN must be reshaped into a 1D sequence that downstream

sequence models (BiLSTM, Transformer, CTC) can consume. *Adaptive pooling* collapses the vertical axis to height 1, leaving a sequence of column vectors that runs left to right across the image. Each column vector summarises a thin vertical strip of the original image and is treated as one “timestep” by the encoder. This trick is what lets a fundamentally 2D convolutional model emit a sequence amenable to CTC.

A.2 VGG16 / VGG19

VGGNets [27] are a family of deep CNNs, developed by Visual Geometry Group (VGG) which showed that by simply adding more layers (depth), the accuracy of image classification can improve. By stacking many small 3×3 convolutions, VGG16 and VGG19 (16 and 19 weight layers respectively) became standard pretrained backbones because of their simple, uniform structure.

A.3 ResNet50 / ResNet101

ResNet [28] introduced residual "skip" connections that let each layer learn a correction to the identity mapping rather than a full transformation. This enables stable training of much deeper networks. ResNet50 and ResNet101 have 50 and 101 weight layers respectively, avoiding the gradient vanishing problems that occur in very deep CNNs.

A.4 ImageNet Pretraining and Transfer Learning

The VGG and ResNet backbones used in this pipeline ship with *ImageNet pretrained* weights, meaning they have already been trained on the ImageNet ILSVRC subset (1.28M images across 1000 categories) that torchvision uses for its pretrained classification models. *Transfer learning* reuses these weights as initialisation for a different downstream task. Like here, for OCR on North Sámi line images.

A.5 BiLSTM

A bidirectional Long Short Term Memory (BiLSTM) [29] network is a type of Recurrent Neural Network (RNN) that runs two LSTMs over the column features, one left to right, the other right to left, and concatenates their hidden states at each position. The RNN is a type of neural network for sequential data processing. They have a hidden state that is updated at each timestep based on the current input and the previous hidden state, allowing them to capture temporal dependencies. However, normal RNNs suffer from the vanishing gradient problem, which makes it difficult to learn long range dependencies.

The LSTM cell uses learned input, forget, and output gates to retain long range dependencies while suppressing irrelevant context, mitigating the vanishing gradient problem, which is well suited to OCR: predicting the character in a column often benefits from knowing both what came before and what is coming next, for example to disambiguate visually similar diacritics by their typical neighbouring characters in Sámi morphology.

A.6 Transformer Encoder

A Transformer encoder [30] replaces recurrence with self attention: every position attends to every other position in parallel, weighted by learned compatibility scores. This gives a fully global receptive field over the column features and removes the sequential bottleneck of recurrent models. In the pipeline, the Transformer encoder serves as a drop in alternative to the BiLSTM, isolating whether explicit global attention buys more than the LSTM’s local bidirectional context for the relatively short column sequences produced by the backbone.

A.7 TrOCR

TrOCR [31] is a transformer based OCR architecture using a Vision Transformer (ViT) encoder paired with an autoregressive text generation decoder. Rather than treating the image as a sequence of column features (the CRNN approach used by this pipeline), TrOCR splits the image into a grid of fixed size patches and processes them with self attention end to end. The decoder then generates output text token by token, conditioned on the encoded image and on its own previous outputs.

A.8 Connectionist Temporal Classification (CTC)

Connectionist Temporal Classification (CTC) [23] is a loss function and decoding scheme for sequence labelling tasks in which the alignment between input frames and output labels is unknown. CTC introduces a special *blank* token and marginalises over all alignments that, after collapsing repeated tokens and removing blanks, produce the target transcription. This removes the need for explicit character level segmentation of the input image, exactly the property needed for variable length OCR lines. At inference time, greedy decoding picks the most likely token at each timestep and applies the same collapse rule to produce the final transcription.

A.9 AdamW Optimizer

AdamW is the optimiser used across all training runs in this pipeline. Adam, the underlying algorithm, adapts the per parameter learning rate using running averages of recent gradients (a momentum term) and their squared magnitudes (a scale term). This makes it robust on noisy gradients and forgiving of poor initialisation, which is why it has become the de facto default for modern deep learning. The “W” variant detaches the weight decay from the adaptive scaling.

A.10 Learning Rate (LR), LR Schedulers and Weight Decay

The *learning rate* controls the step size taken along the gradient direction when updating weights of the model.

Differential learning rates. Apply different LR multipliers to different parameter groups inside the same optimiser.

LR schedulers. Change the learning rate over the course of training.

`ReduceLROnPlateau`, monitors the validation CER and halves the learning rate whenever it stops improving for a fixed number of epochs, so the network starts in an aggressive regime and gradually settles into a finer grained one without manual intervention. *Warmup* works at the other end of training: starting from a tiny learning rate and ramping up over the first few epochs, which prevents large early gradients from destabilising pretrained weights.

Weight decay. A regulariser that adds a small penalty proportional to the squared magnitude of each weight, gently pulling the network toward simpler solutions. This discourages overfitting and keeps unused weights from drifting to large arbitrary values that do not help generalisation.

Gradient clipping. Caps the global norm of the gradient vector at a fixed threshold. CTC losses are prone to occasional spikes when the model is briefly very wrong about an alignment. Clipping it stops a single bad batch from blowing up the weights.

A.11 Epochs, Batch Size, Training Steps, and Early Stopping

An *epoch* is one complete pass over the training set. The *batch size* is the number of samples whose gradients are averaged before each weight update.

A *training step* is a single weight update, one batch processed. The number of steps per epoch is therefore the dataset size divided by the batch size.

Early stopping halts training when the validation metric has stopped improving for `patience` epochs, restoring the model from the best checkpoint observed.

A.12 Data Augmentation

Data augmentation expands the effective training set by applying random transformations to each sample on the fly: minor rotations, scale jitter, brightness and contrast perturbations, Gaussian blur, and synthetic noise. That way we can synthetically generate a more diverse set of training images.

A.13 OCR Evaluation Metrics

OCR quality is measured against a reference transcription using string edit metrics. The *Levenshtein distance* between two strings is the minimum number of single character insertions, deletions, or substitutions needed to turn one into the other.

Character Error Rate (CER). The Levenshtein distance between the predicted and reference strings, divided by the length of the reference in characters. The lower the better.

Word Error Rate (WER). The same idea but at the word level: edit distance counted in word tokens rather than characters, divided by the reference length in words. WER is harsher than CER, because a single misrecognised character can corrupt an entire word.

Exact Match Accuracy. The fraction of test samples for which the predicted transcription matches the reference verbatim. A binary, precision metric.

Normalised Accuracy. Exact match accuracy computed after a normalisation pass (removing punctuation, collapsing repeated whitespace). Useful for separating substantive recognition errors from cosmetic formatting differences.

A.14 Machine Translation Metrics

n gram. A contiguous sequence of `n` tokens. For example, in the sentence “the cat sat”, the 1 grams are “the”, “cat”, and “sat”, while the 2 grams are “the

cat” and “cat sat”. MT metrics often operate on n grams to measure how well the candidate translation matches the reference.

BLEU. Bilingual Evaluation Understudy. Measures the n gram precision of the candidate translation against one or more references, multiplied by a brevity penalty that discourages overly short outputs. Widely used. Might be coarse for morphologically rich languages where surface forms vary heavily.

chrF. Character n gram F score [25]. Operates on character n grams instead of word n grams, which makes it more forgiving of morphological variation.

TER. Translation Edit Rate: the number of edits (insertions, deletions, substitutions, shifts) needed to turn the candidate into the reference, normalised by the reference length. Closer to zero is better.

A.15 Statistical Significance and Effect Size

Percentage points (pp). The arithmetic difference between two percentage values, distinct from a relative percentage change.

Paired t test. A hypothesis test for whether the mean difference between two paired sets of observations is zero. “Paired” means each prediction in condition A has a one to one counterpart in condition B.

p value. The probability of observing a result at least as extreme as the measured one if the null hypothesis (no true difference) were true. A small p value (conventionally $p < 0.05$) is read as evidence against the null.

Effect size. A standardised measure of *how large* a difference between two conditions is, independent of sample size. A p value can be tiny merely because the sample is large, even if the underlying effect is negligible, effect sizes report the gap on a scale that is interpretable in its own right.

Cohen’s d . A common effect size statistic for the difference between two means: the mean difference between two conditions divided by the pooled standard deviation. It expresses the gap in units of typical variability, so it is comparable across studies and immune to sample size inflation. Conventional rules of thumb: $d \approx 0.2$ small, 0.5 medium, 0.8 large.

B Appendix

B.1 Code and Models

1. Source code (GitHub): <https://github.com/magwrap/north-sami-ocr/>
2. Trained lightweight CNN-CTC model card (HuggingFace): <https://huggingface.co/magwrap/cnn-ctc-ocr-sme>
3. Språkbanken synthetic Sámi OCR dataset (HuggingFace, mirrored): https://huggingface.co/datasets/magwrap/synthetic_sami_ocr_data

B.2 Conference Submissions

1. **IISA 2026**, International Conference on Information, Intelligence, Systems and Applications. Full paper accepted to the *21st century AI and education* theme. Venue: <https://easyconferences.eu/iisa2026/>.
2. **SCAI 2026**, 15th Scandinavian Conference on Artificial Intelligence. Extended abstract accepted to the *AI for Good: Innovation, Ethics, Society* theme. Venue: https://www.sdu.dk/en/forskning/sdu_applied_ai_and_data_science/scai.

B.3 Supervisor and Meeting Log

Weekly meetings were held on Tuesdays, serving as the primary supervisory mechanism throughout the project. These meetings provided a structured opportunity for regular check ins, progress updates, and feedback sessions. The discussions during these meetings were instrumental in guiding the research direction, troubleshooting issues, and refining methodologies.

B.4 AI usage

This declaration follows SDU's guidelines on generative AI in student work, which require transparent disclosure of how AI contributed to the final product. Throughout the project, generative AI tools, namely Claude Code (Sonnet 4.6, Opus 4.7, Opus 4.8) were utilized for the following categories of tasks:

- **Programming and code generation:** AI assisted with writing code snippets, boilerplate, and data processing scripts. Example: the alignment tool experiment for parallel corpora, which would normally require weeks of development time, was prototyped in hours with AI assistance, freeing time for experimentation and analysis. All AI-generated code was read, tested, and modified before integration.
- **Brainstorming and ideation:** AI was used to generate ideas, suggest alternative approaches, and surface considerations that a less experienced researcher might overlook. All directions explored were subsequently verified against the literature and the supervisor’s feedback.
- **Language proofreading:** AI was used to check grammar, spelling, punctuation, and to improve clarity of phrasing. AI-suggested rewordings were reviewed and edited before integration.
- **Structure and organization:** AI was occasionally consulted on chapter and section ordering and on how to frame arguments.
- **Figure generation:** AI assisted in producing the Python/matplotlib scripts used to generate figures and LaTeX tables (see `figures/scripts/`). The underlying data and the analytical claims drawn from there were independent of AI.

Transparency and accountability. No AI-generated text or output is presented in this report as the my own work without disclosure. I have critically evaluated every AI contribution for relevance, accuracy, and factual correctness, and take full responsibility for the final content of the report.

Bibliography

- [1] Lars Ivar Hansen and Bjørnar Olsen. *Hunters in Transition: An Outline of Early Sámi History*. Brill, Leiden, 2014.
- [2] Håkan Rydving. *The End of Drum-Time: Religious Change among the Lule Saami, 1670s–1740s*. Acta Universitatis Upsaliensis, Uppsala, 1993.
- [3] Henry Minde. Assimilation of the Sami: Implementation and consequences. *Gáldu Čála: Journal of Indigenous Peoples Rights*, (3), 2003.
- [4] Patrik Lantto. Borders, citizenship and change: The case of the Sami people, 1751–2008. *Citizenship Studies*, 14(5):543–556, 2010.
- [5] Veli-Pekka Lehtola. *The Sámi People: Traditions in Transition*. University of Alaska Press, Fairbanks, 2004.
- [6] Lukas Allemann. *The Sami of the Kola Peninsula: About the Life of an Ethnic Minority in the Soviet Union*. PhD thesis, Centre for Sami Studies, UiT The Arctic University of Norway, Tromsø, 2013.
- [7] Eva Josefsen, Ulf Mörkenstam, and Jo Saglie. Different institutions within similar states: The Norwegian and Swedish Sámediggis. *Ethnopolitics*, 14(1):32–51, 2015.
- [8] Sannhets- og forsoningskommisjonen. Sannhet og forsoning – grunnlag for et oppgjør med fornorskingspolitikk og urett mot samer, kvener/norskfinner og skogfinner. Technical report, Stortinget, Oslo, 2023. Truth and Reconciliation Commission report, delivered 1 June 2023.
- [9] Stortinget. Resolutions in response to the truth and reconciliation commission’s report, 2024. Formal apology to the Sámi, Kven/Norwegian Finn, and Forest Finn peoples, adopted November 2024.
- [10] Tita Enstad, Trond Trosterud, Marie Iversdatter Røsok, Yngvil Beyer, and Marie Roald. Comparative analysis of optical character recognition methods for Sámi texts from the National Library of Norway. In *Proceedings of the Joint 25th Nordic Conference on Computational Linguistics and 11th Baltic Conference on Human Language Technologies (NoDaLiDa/Baltic-HLT)*, pages 98–108, 2025.
- [11] The Decolonial Atlas. Sápmi – the sámi homelands. <https://decolonialatlas.wordpress.com/2021/02/08/>

- sapmi-the-sami-homelands/, 2021. Accessed May 2026. Reused under the Decolonial Media License 0.1.
- [12] Pekka Sammallahhti. *The Saami Languages: An Introduction*. Davvi Girji, Kárášjohka/Karasjok, 1998.
 - [13] Christopher Moseley, editor. *Atlas of the World's Languages in Danger*. UNESCO Publishing, Paris, 3 edition, 2010.
 - [14] Riitta-Liisa Valijärvi and Lily Kahn. *North Sámi: An Essential Grammar*. Routledge, London, 2017.
 - [15] Johan Turi. *Muitalus sámiid birra*. 1910. English translation: T. A. DuBois, *An Account of the Sámi*, Nordic Studies Press, Chicago, 2011.
 - [16] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 8024–8035, 2019.
 - [17] Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, Mariama Drame, Julien Plu, Lewis Tunstall, Joe Davison, Mario Šaško, Gunjan Chhablani, Bhavitvya Malik, Simon Brandeis, Teven Le Scao, Victor Sanh, Canwen Xu, Nicolas Patry, Angelina McMillan-Major, Philipp Schmid, Sylvain Gugger, Clément Delangue, Théo Matussière, Lysandre Debut, Stas Bekman, Pierric Cistac, Thibault Goehringer, Victor Mustar, François Lagunas, Alexander M. Rush, and Thomas Wolf. Datasets: A community library for natural language processing. In *Proc. Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, pages 175–184, 2021.
 - [18] Matt Post. A call for clarity in reporting BLEU scores. In *Proc. Third Conference on Machine Translation: Research Papers (WMT)*, pages 186–191, 2018.
 - [19] Giellatekno. Giellatekno: Center for Sámi language technology, UiT The Arctic University of Norway. <https://giellatekno.uit.no/>, 2025.
 - [20] Språkbanken, National Library of Norway. Sámi OCR models collection. <https://huggingface.co/collections/Sprakbanken/sami-ocr>, 2025.

- [21] Språkbanken, National Library of Norway. Synthetic Sámi OCR data. https://huggingface.co/datasets/Sprakbanken/synthetic_sami_ocr_data, 2025.
- [22] Baoguang Shi, Xiang Bai, and Cong Yao. An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Trans. Pattern Anal. Mach. Intell.*, 39(11):2298–2304, 2017. arXiv:1507.05717.
- [23] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proc. ICML*, pages 369–376, 2006.
- [24] TartuNLP. Neural machine translation API. University of Tartu, 2025. <https://api.tartunlp.ai/translation/v2>.
- [25] Maja Popović. chrF: Character n-gram F-score for automatic MT evaluation. In *Proc. WMT*, pages 392–395, 2015.
- [26] Sjur Nørstebø Moshagen, Trond Trosterud, and Linda Wiecheteck. Building an open-source development infrastructure for language technology projects. In *Proc. LREC*, pages 859–866, 2014.
- [27] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *Proc. ICLR*, 2015.
- [28] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proc. IEEE CVPR*, pages 770–778, 2016.
- [29] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proc. NeurIPS*, pages 5998–6008, 2017.
- [31] Minghao Li, Tengchao Lv, Jingye Chen, Lei Cui, Yijuan Lu, Dinei Florencio, Cha Zhang, Zhoujun Li, and Furu Wei. TrOCR: Transformer-based optical character recognition with pre-trained models. In *Proc. AAAI*, volume 37, pages 13094–13102, 2023.